

Week 4 / 6

# Talking to Claude Code for Git

and design-pr-review v2

---

GitHub for UI Designers  
Karen Shen  
May26, 2026

## AGENDA

# What we'll cover today

---

A

## Talking to Claude Code about Git

*Six topics × Say / Don't say*

- Three-week topic map
- Six topics × Say / Don't say
- Phrases to always avoid + safety zones

B

## design-pr-review skill

*deep dive*

- **Three-stage breakdown + Demo**
  - **Handoff to pr-comment**
  - **Battle-tested cases + boundaries**
-

*GitHub operation*

# Talking to Claude Code

*From concepts to AI-friendly phrasing*

---

Extension of Week 2 #17 + #21 — Six topics × Say / Don't say

# What we've covered

*Three-week topic map*

---

## Week 1

### What is Git + why designers care

Git / GitHub basics

Why version control

---

## Week 2

### Working inside a repo

Clone / Setup

Branch / Switch

Push / Pull / Reject

---

## Week 3

### Git vocabulary + Working repo types

Commit vs Push

Staged / Stash / Discard

PR / Merge / Conflict

# Cloning and connecting a repo

**Rule: name the repo / file / branch explicitly. Verbs must be precise (clone / fork / init are different).**

Bad phrasing sends AI down a different path — the consequence column shows what actually goes wrong.

| What you want                                                   | Say this                                                                                   | Don't say + consequence                                                                                                                                                                                |
|-----------------------------------------------------------------|--------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>把 repo 抓下來</b><br><i>Clone a repo</i>                        | ✓ 「clone trendlife-general/hie-rei」<br><i>"clone trendlife-general/hie-rei"</i>            | ✗ 「下載這個」<br><i>"download this"</i><br><div>⚠ AI may trigger GitHub's web Download ZIP — you get a zip with no .git, no remote. commit / push / pull all fail.</div>                                    |
| <b>把本機資料夾接到 GitHub</b><br><i>Connect local folder to GitHub</i> | ✓ 「把資料夾連到 GitHub 上的 <org/repo>」<br><i>"connect this folder to GitHub &lt;org/repo&gt;"</i> | ✗ 「設成我的 repo」<br><i>"make it my repo"</i><br><div>⚠ "Make it" → AI guesses fork / init / rename. "My" → personal GitHub or local? Worst case: fork the wrong repo, or git init clobbers history.</div> |
| <b>指代陷阱（全主題通用）</b><br><i>Reference traps (any topic)</i>        | ✓ repo / branch / 檔名直接念出來<br><i>Spell out repo / branch / file names</i>                   | ✗ 「這個 / 那個 / 連結這個 repo」<br><i>"this / that / link this repo"</i><br><div>⚠ AI grabs whichever repo was last mentioned in chat — likely not the one you mean now.</div>                                 |

# Branching and switching

*Open, switch, check, delete*

**Rule: spell out branch names. Say local or remote.**

| What you want                                      | Say this                                                                                      | Don't say + consequence                                                                                   |
|----------------------------------------------------|-----------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <b>開新 branch 開始工作</b><br><i>Start a new branch</i> | ✓ 「開一條 branch 叫 feature/header-redesign」<br><i>"open branch feature/header-redesign"</i>      | ✗ 「開個分支」（沒名字 AI 會自己編）<br><i>"open a branch" (no name — AI invents one)</i><br><div>In HIE-REI hooks</div> |
| <b>切到別的 branch</b><br><i>Switch branches</i>       | ✓ 「切到 main」<br><i>"switch to main"</i>                                                        | ✗ 「換到那個」「切回去」（指代不清）<br><i>"switch to that one" / "switch back" (ambiguous)</i>                            |
| <b>看目前在哪</b><br><i>Check current branch</i>        | ✓ 「我現在在哪條 branch？」<br><i>"which branch am I on?"</i>                                          | ✗ 「看一下狀況」（範圍太廣）<br><i>"look at the situation" (too broad)</i>                                             |
| <b>刪掉不要的 branch</b><br><i>Delete a branch</i>      | ✓ 「刪掉 feature/abandoned 這條 branch（local）」<br><i>"delete branch feature/abandoned (local)"</i> | ✗ 「清掉舊的」「整理一下」（不知刪哪條）<br><i>"clean up the old ones" / "tidy up" (which one?)</i>                          |

# Recording changes and pushing

Commit and push are two different actions — vague words make AI do something else; worst case, lose changes or push to the wrong branch.

**Rule: commit / push / pull / log are separate verbs. Be precise so AI can't guess the scope.**

What you want

Say this

Don't say + consequence

把改動 commit

*Commit changes*

✓ 「把這次改動 commit, message : <x>」

*"commit this with message: <x>"*

✗ 「存檔」

*"save it"*

⚠ AI may run `git stash` (changes go to the stash, no commit record) or only `git add`. You think it's pushed; the remote sees nothing.

推上 GitHub

*Push to remote*

✓ 「push 到我這條 branch」

*"push to my branch"*

✗ 「上傳 / 同步上去」

*"upload / sync it up"*

⚠ AI may run `git push --all` (push every branch), push to the wrong remote, or add `--force` on conflict. Worst case: WIP with secrets pushed to a public repo.

拉同事的最新版

*Pull latest*

✓ 「拉一下 main 最新的」

*"pull latest main"*

✗ 「更新一下」

*"update it"*

⚠ AI may run `git pull --rebase` (rebase your unpushed commits), or `git reset --hard origin/main` (local uncommitted changes gone, unrecoverable).

# Controlling the scope of a commit

This group is the easiest to wreck — "clean / reset" deletes changes. Always be explicit about scope.

**⚠ Rule: "discard / clear" permanently deletes changes — commit or stash before saying it.**

| What you want                                         | Say this                                                                    | Don't say + consequence                                                          |
|-------------------------------------------------------|-----------------------------------------------------------------------------|----------------------------------------------------------------------------------|
| <b>只把某幾個檔加進 commit</b><br><i>Stage specific files</i> | ✓ 「把 src/foo.tsx 加進這次 commit」<br><i>"stage src/foo.tsx for this commit"</i> | ✗ 「加進去」「全選」（哪個？）<br><i>"add it" / "select all" (which?)</i>                      |
| <b>丟掉某個檔的改動</b><br><i>Discard a file's changes</i>    | ✓ 「丟掉 settings.html 的改動」<br><i>"discard changes to settings.html"</i>       | ✗ 「reset」「清掉」「回到原本」（範圍太大）<br><i>"reset" / "clear it" / "go back" (too broad)</i> |
| <b>暫存目前狀態等等回來</b><br><i>Stash work-in-progress</i>    | ✓ 「把目前狀態 stash 起來等等回來」<br><i>"stash this and come back to it"</i>           | ✗ 「先放著」「先存著」（會被當 commit）<br><i>"keep it aside" / "save it for now"</i>           |
| <b>拿回 stash 的東西</b><br><i>Restore stashed work</i>    | ✓ 「把剛剛 stash 的拿回來」<br><i>"bring back the last stash"</i>                    | ✗ 「之前那個」（哪一個 stash？）<br><i>"that earlier one" (which stash?)</i>                 |



# Pull requests, merging, conflicts

*Name the verb you actually mean*

"Merge" is ambiguous — branch merge? PR merge? squash? rebase? Always name the action.

## What you want

## Say this

## Don't say + consequence

### 開 PR

*Open a pull request*

✓ 「開 PR 把 feature/x 合進 main, title: <x>」

*"open PR from feature/x into main, title: <x>"*

✗ 「送出去 review」 (送哪?)

*"send it out for review" (send where?)*

### 把 main 最新的併進來

*Bring main into branch*

✓ 「幫我把 PR #310 rebase 到 main」

*"rebase PR #310 onto main"*

✗ 「同步一下」「更新分支」 (rebase 還 merge?)

*"sync it up" / "update the branch" (rebase or merge?)*

### 解 conflict

*Resolve a conflict*

✓ 「解這個 conflict, 保留我這邊 (ours)」

*"resolve this conflict, keep mine (ours)"*

✗ 「處理一下」 (保留誰?)

*"deal with it" (keep whose?)*

### PR merge 進 main

*Merge the PR*

✓ 「PR #310 squash merge」

*"squash merge PR #310"*

✗ 「合進去」 (squash? rebase? merge commit?)

*"merge it in" (squash? rebase? merge commit?)*

# Three repo types, three ways to talk

*Different repo, different phrasing*

**Rule: identify which type of repo you're in before acting. Decide whether a PR is required.**

What you want

Say this

Don't say + consequence

**A 協作 repo (你有寫入)**

*A Collaboration (you can write)*

✓ 「開 PR 把 <branch> 合進 main」  
"open a PR to merge <branch> into main"

✗ 「直接 push 到 main」 (會被擋)  
"push directly to main" (blocked)

In HIE-REI hooks

**B 引用 repo (你是讀者)**

*B Reference (DS, guidelines)*

✓ 「同步最新的 design system tokens」  
"sync the latest design system tokens"

✗ 「我要改」 (沒寫入權限, AI 還是會試)  
"I want to edit" (no write access — AI tries anyway)

**C 個人 repo**

*C Personal repo*

✓ 「commit + push, message : <x>」  
"commit + push, message: <x>"

✗ — (個人 repo 約束最少)  
— (personal repos have the fewest constraints)

**搞不清楚是哪一種**

*Not sure which type*

✓ 「我有這個 repo 的寫入權限嗎？」  
"do I have write access to this repo?"

✗ 直接動手 (可能踩權限或 review 規範)  
Just acting — may trip permissions / review rules

# Phrases that trigger destructive actions

*Words that map to --force / reset --hard / rm -rf*

These phrases make AI run --force / reset --hard / rm -rf — unrecoverable.

## Phrases to avoid

✗ "force push it"

✗ "just push it"

✗ "just overwrite"

✗ "start over"

✗ "delete everything"

✗ "--force / --force-with-lease"

## Why it breaks — force push consequence

Before:



(▲ = teammate's just-pushed commit)

**AI force push ↓**

After:



(▲ gone — overwritten by your commits)

**Teammate's commit vanishes from remote — only reflog can save it**

# Auto-run vs Confirm-first (Will be added to project hooks)

*Reversibility decides who pulls the trigger*

Reversibility decides whether to pause — irreversible actions always need confirmation.

## ✓ Let AI run it

*Safe — read-only or trivially reversible*

- ``git status``
- ``git diff`` / ``git diff --staged``
- ``git log`` / ``git show``
- ``git branch -l`` / ``gh pr list``
- ``git fetch`` (no working dir change)
- ``gh pr view <num>``

## ⚠ Confirm first

*Destructive or hard-to-reverse*

- ``git push`` / ``git push --force-with-lease``
- ``git reset --hard`` / ``git clean -fd``
- ``git rebase --onto`` / wide rebase
- ``git branch -D`` / ``git tag -d``
- ``rm -rf`` / file delete
- ``gh pr merge`` / any merge action

# design-pr-review skill

*Automating the design PR review steps*



Scope → Side-by-side → Violation flag + Multi-author scan

# How to trigger the skill

Already built into hie-rei main — no install. Just trigger it.

---

## Slash command

```
/design-pr-review <PR-number>
```

## Natural language

```
review this design PR #321
```

## Casual phrasing

```
help me review the design in PR #321
```

### Prerequisites

- Your Claude Code has hie-rei cloned and checked out to main
- `gh auth login` with read access to the target repo
- The PR is a design PR (HTML mockup + spec.md)

# Three things the skill does

*Three stages, one click*

Each stage returns its output to you so you decide what's next — you stay in the driver's seat.

---

1

## Scope the PR

Auto-list changed files. Identify mockup vs spec.md. Filter unrelated config files.

→ *a list of changed files with each file's role labelled*

---

2

## Before / After Side-by-side

Open two browser tabs automatically — main version on the left, PR version on the right.

→ *visual diff at a glance*

---

3

## Violation flag + Multi-author scan

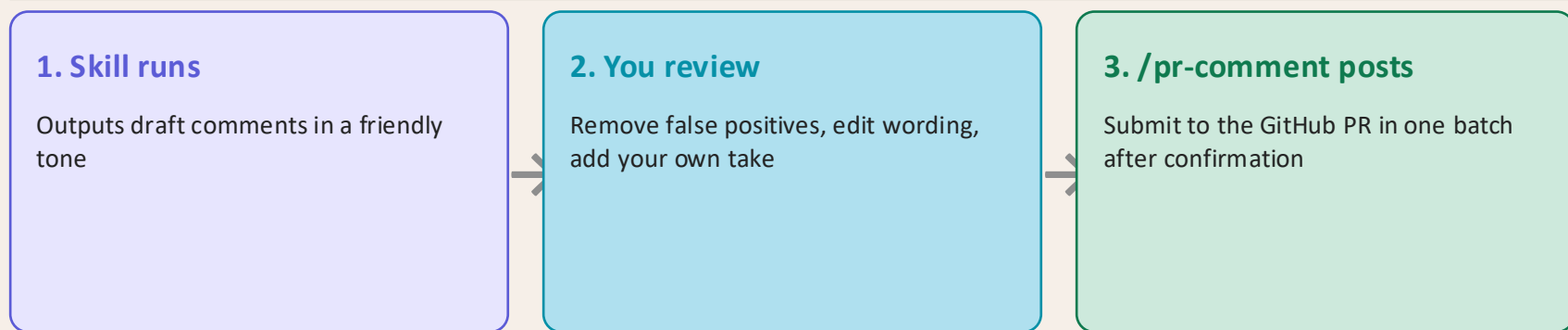
Surface design-system violations as questions (not verdicts). Flag files touched by multiple authors.

→ *material for your review comments*

# design-pr-review → pr-comment handoff

*Draft, review, then post*

design-pr-review only drafts — it never posts. You review, edit, then publish.



## Why two stages?

- The skill is a co-pilot, not the driver — it drafts, but pressing send is your call
- PR comments email the whole team via GitHub — one typo reaches everyone
- Your review = wording stays clean, fits team voice, designer judgment preserved



## KEY TAKEAWAY

# Two threads, one principle

*Hands on the wheel · Eyes on the diff · Mind on the design*



### Speak precisely

Name the repo / file / branch / commit. Never "this" / "that". AI won't guess right.



### Confirm destructive actions

push / reset / rebase / merge / rm — always diff and confirm before pulling the trigger.



### Skill is co-pilot

design-pr-review drafts the material,  
pr-comment lets you publish.  
Judgment and ownership stay with you.